

Отчёт по практическому заданию

Система контроля версий Git

Выполнила: Рыжих Вероника Александровна
Группа 2122

Репозиторий: <https://github.com/00Veronika00/git-practice>
Дата выполнения: 28.09.2026

Шаг 1. Создание локального репозитория

Проверили, что Git установлен (**git --version**), указали имя и адрес электронной почты (они записываются в каждый коммит), затем в папке проекта выполнили **git init**: эта команда создаёт скрытый каталог .git со структурой репозитория. Команда **git status** показывает «No commits yet»: репозиторий пуст, коммитов ещё нет. Адрес почты на скриншоте скрыт.

```
git --version
git config --global user.name "Рыжих Вероника"
git config --global user.email <почта>
git init
git status
```

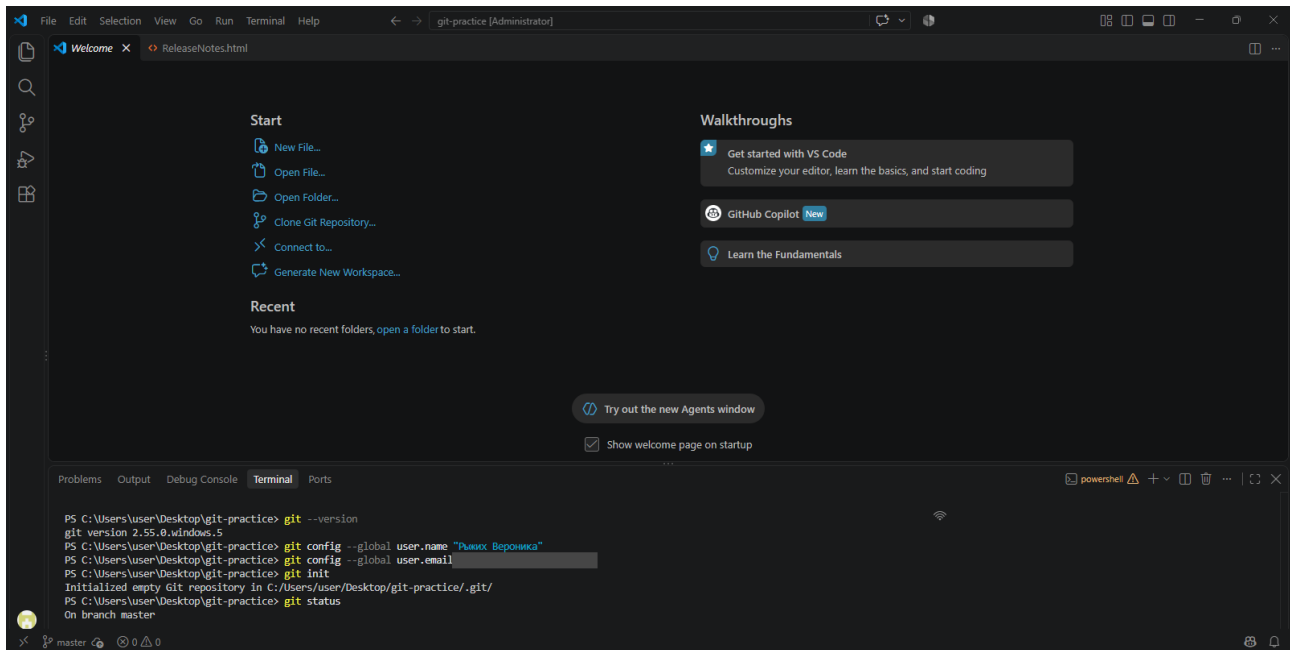


Рис. 1. Инициализация репозитория, ветка master, коммитов пока нет

Шаг 2. Два файла и первый коммит

В папке проекта созданы два текстовых файла: file1.txt («Первый файл») и file2.txt («Второй файл»). В VS Code они отмечены буквой U (untracked), то есть Git о них пока не знает. Командой **git add** файлы добавляются в индекс (staged, буква A), а **git commit** фиксирует их в истории. После коммита **git status** сообщает «working tree clean».

```
git status
git add file1.txt file2.txt
git status
git commit -m "first commit"
git status
```

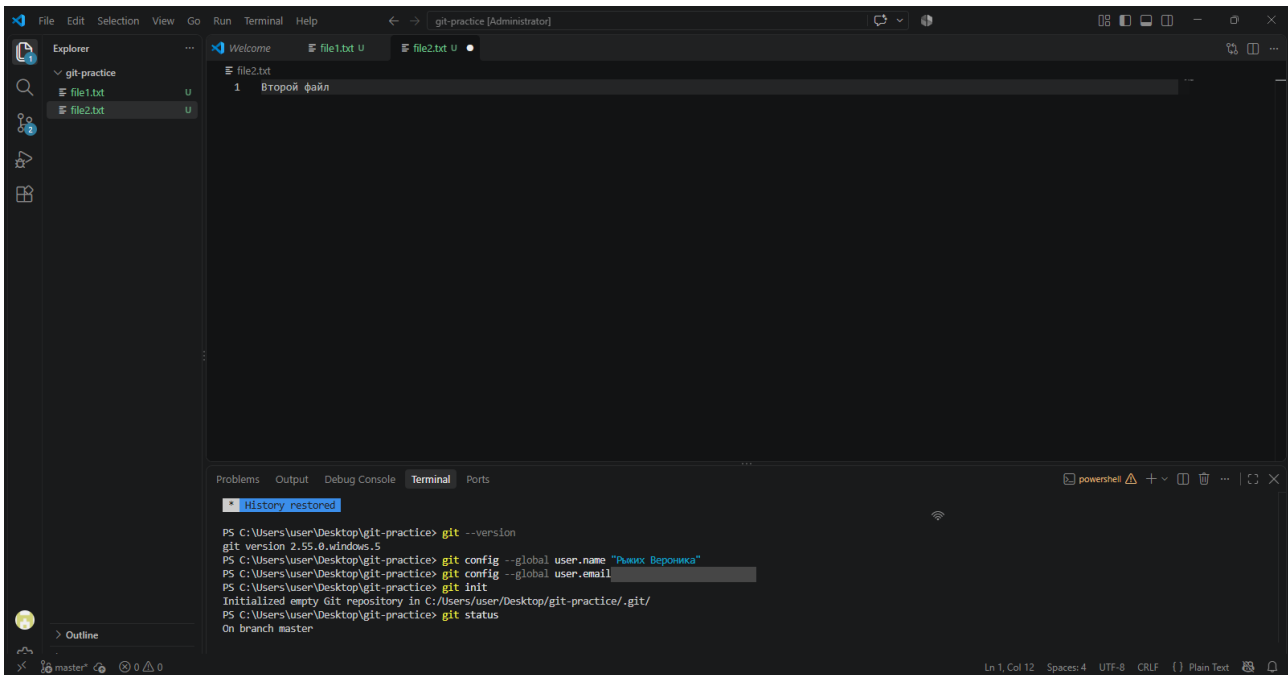


Рис. 2. Файл file2.txt, помечен как неотслеживаемый (U)

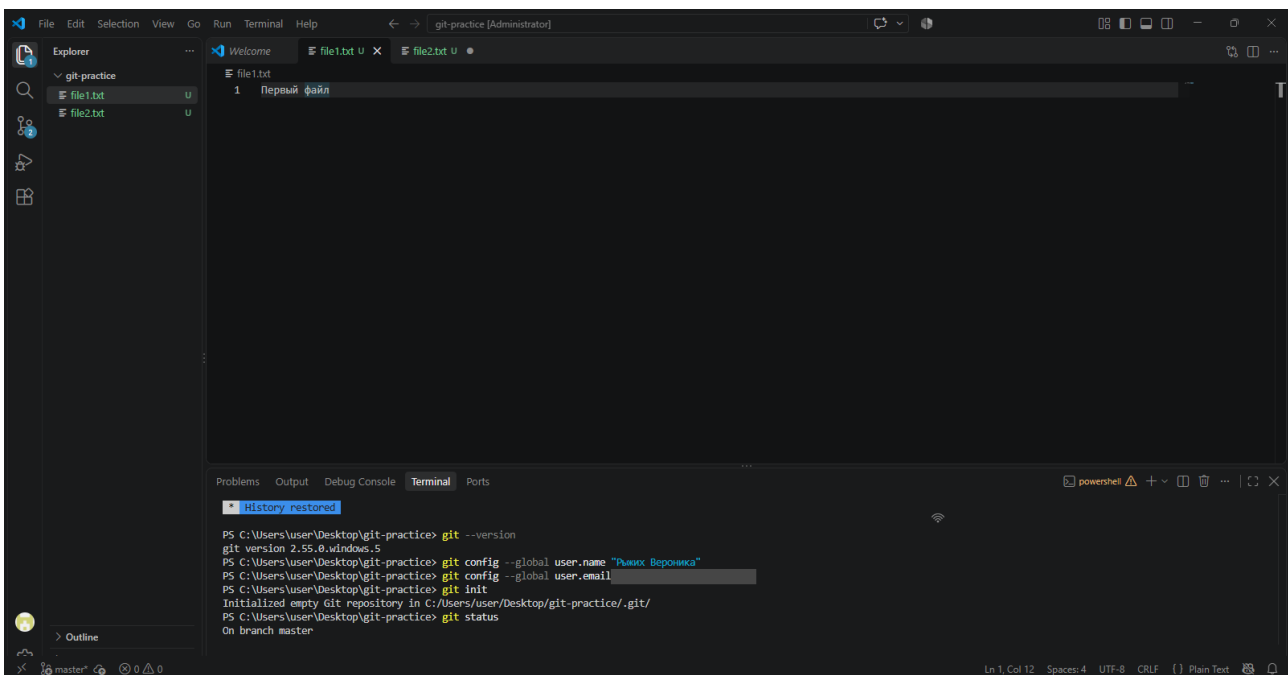


Рис. 3. Файл file1.txt, помечен как неотслеживаемый (U)

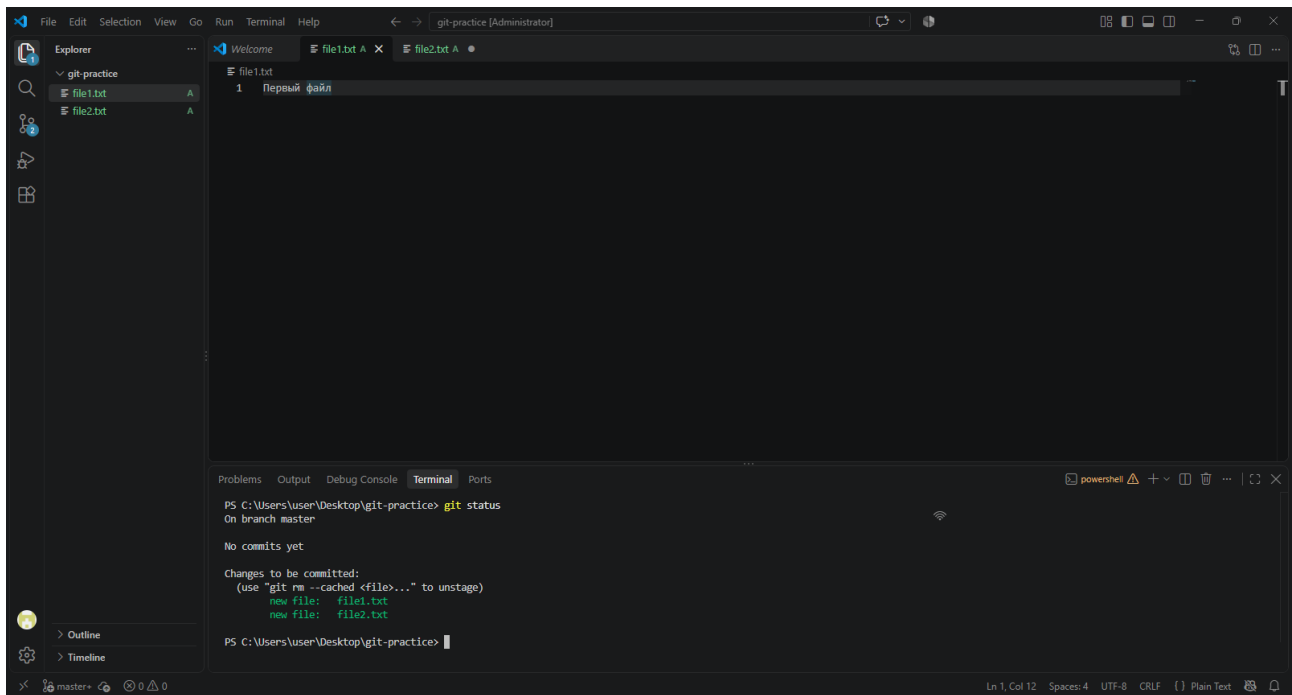


Рис. 4. Файлы добавлены в индекс (staged), «Changes to be committed»

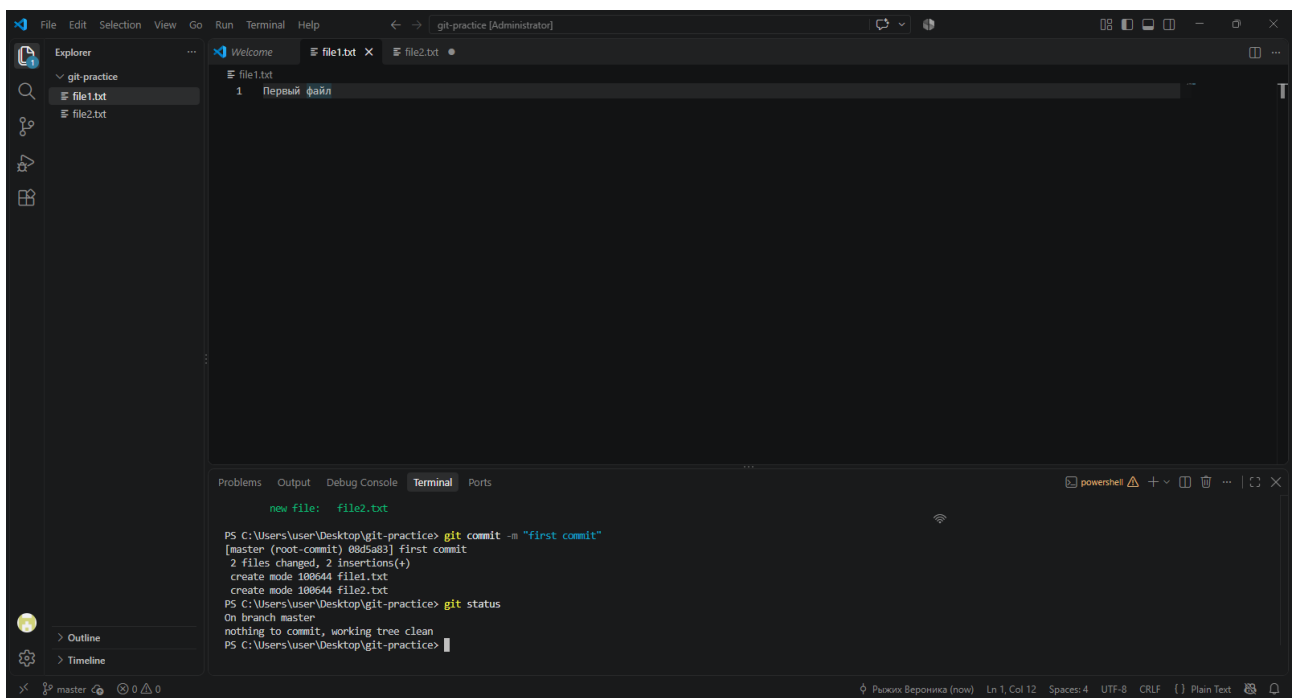


Рис. 5. Первый коммит и чистое рабочее дерево

Шаг 3. Файл .gitignore

Создан файл .gitignore с двумя шаблонами: ***.log** (все файлы с расширением .log) и **temp/** (папка temp). Для проверки создан файл debug.log. В выводе **git status** появляется только .gitignore, а debug.log отсутствует: Git его игнорирует, хотя файл остаётся в папке.

```
*.log  
temp/
```

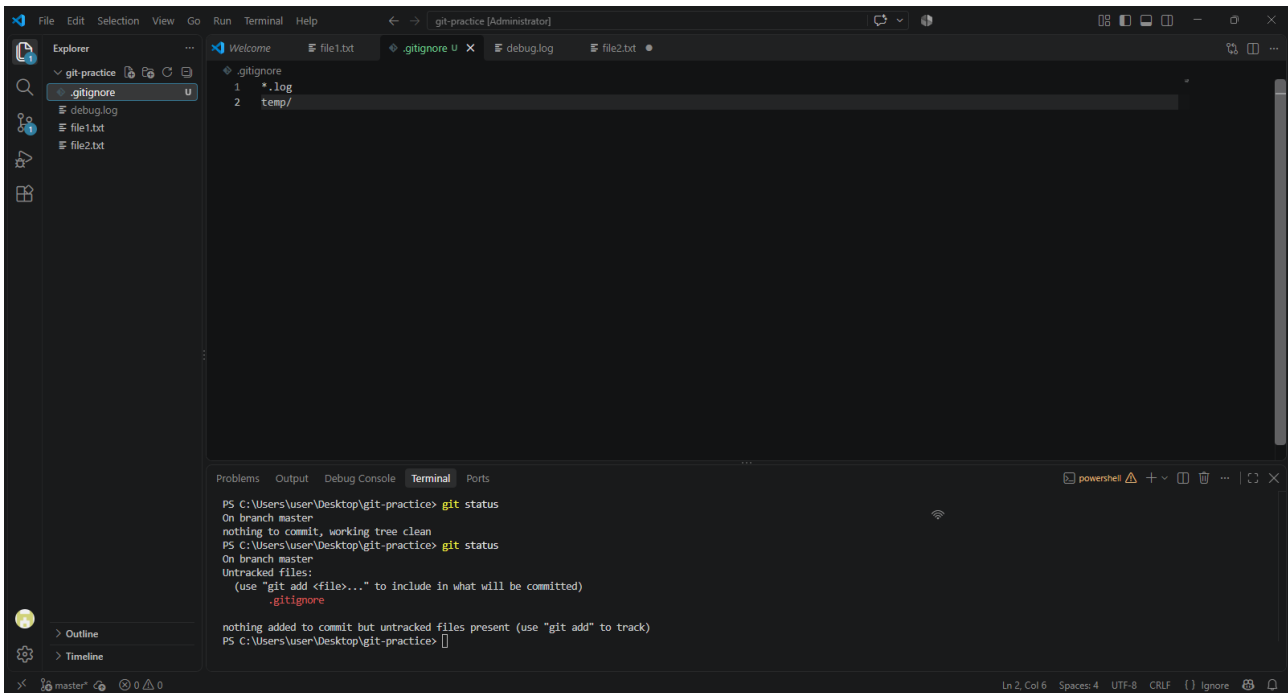


Рис. 6. Содержимое .gitignore; в git status виден только .gitignore

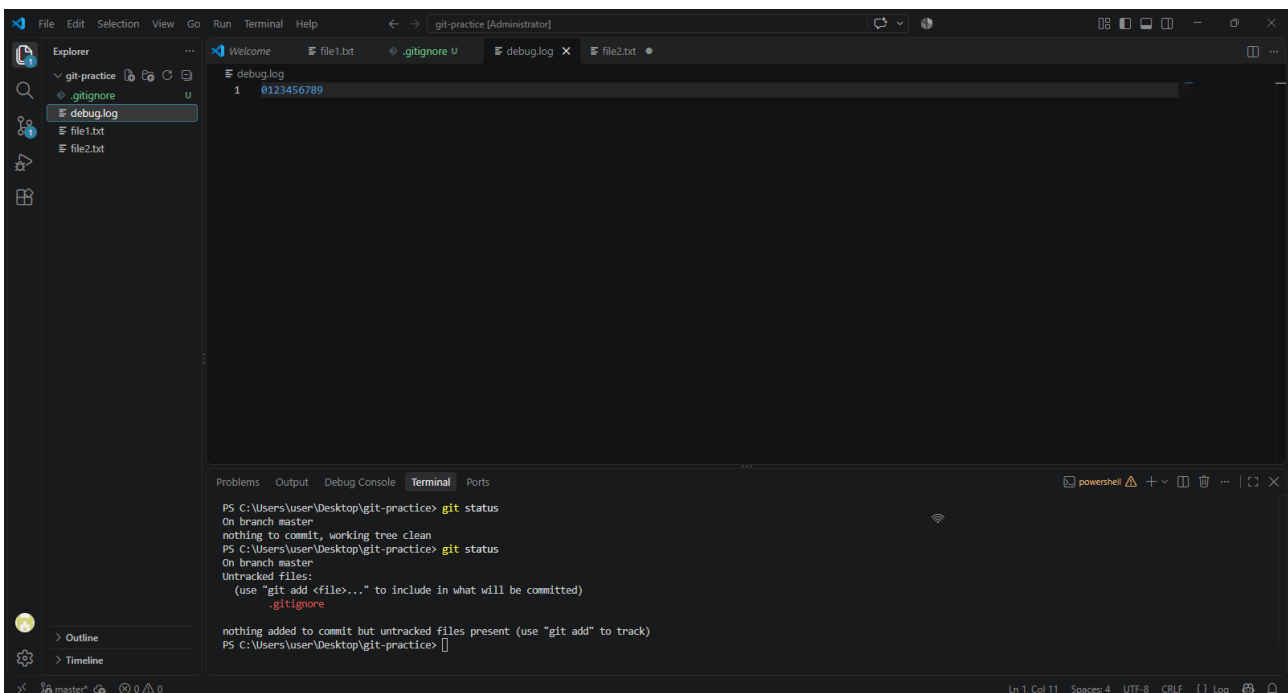
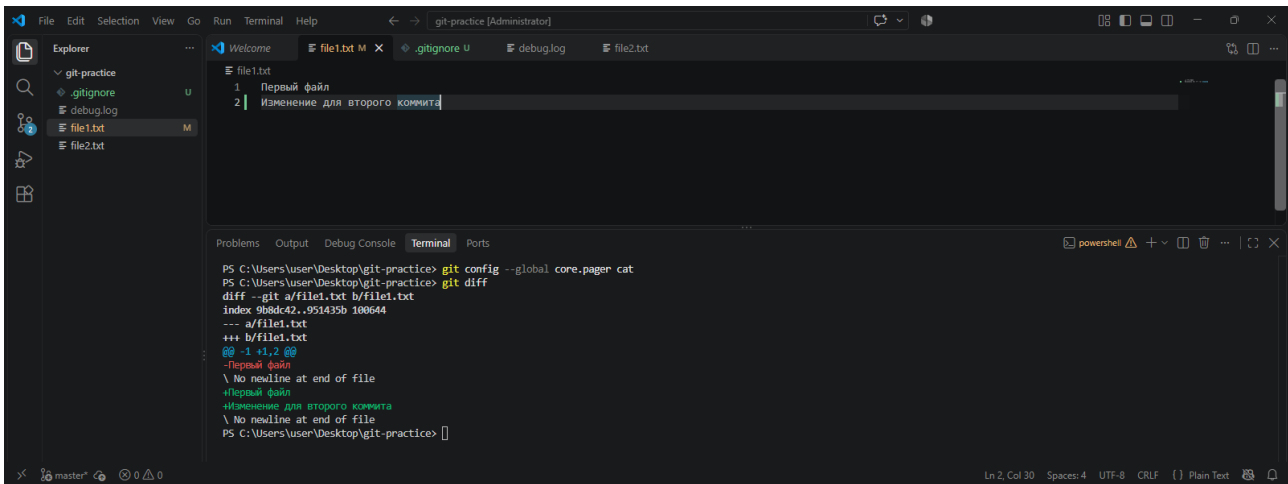


Рис. 7. Файл debug.log существует, но в git status не отображается

Шаг 4. Изменение файла и второй коммит

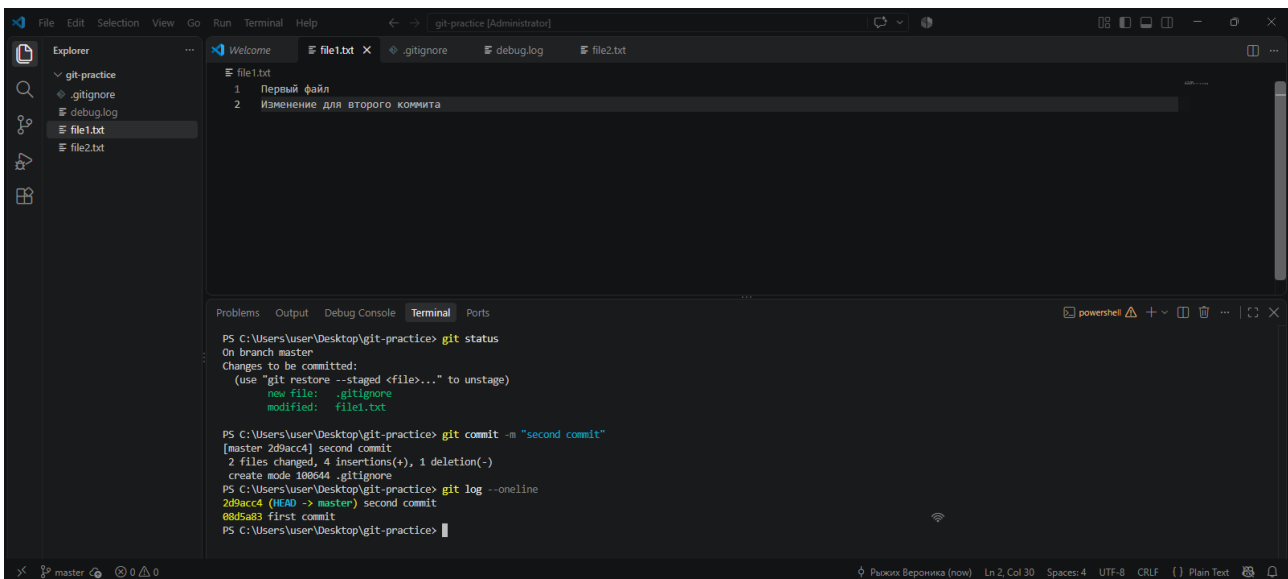
В file1.txt добавлена строка «Изменение для второго коммита». Команда **git diff** показывает разницу с последним коммитом (удалённая строка красным, добавленная зелёным). Затем изменение и файл .gitignore добавлены в индекс и закоммичены. **git log --oneline** показывает два коммита.

```
git diff
git add .
git commit -m "second commit"
git log --oneline
```



```
PS C:\Users\user\Desktop\git-practice> git config --global core.pager cat
PS C:\Users\user\Desktop\git-practice> git diff
diff --git a/file1.txt b/file1.txt
index 9b8dc42..951435b 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1 +1,2 @@
-Первый файл
\ No newline at end of file
+Изменение для второго коммита
\ No newline at end of file
PS C:\Users\user\Desktop\git-practice>
```

Рис. 8. Результат git diff для file1.txt



```
PS C:\Users\user\Desktop\git-practice> git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitignore
    modified:   file1.txt

PS C:\Users\user\Desktop\git-practice> git commit -m "second commit"
[master 2d9acc4] second commit
2 files changed, 4 insertions(+), 1 deletion(-)
create mode 100644 .gitignore

PS C:\Users\user\Desktop\git-practice> git log --oneline
2d9acc4 (HEAD -> master) second commit
0805a03 first commit
PS C:\Users\user\Desktop\git-practice>
```

Рис. 9. Второй коммит и история из двух коммитов

Шаг 5. Новая ветка и коммит в ней

Командой **git checkout -b feature** создана ветка feature и выполнено переключение на неё (название ветки видно внизу слева в VS Code). В ветке изменён другой файл, file2.txt (добавлена строка «Изменение в ветке feature»), и сделан коммит. Лог с графом показывает, что ветка feature ушла на один коммит вперёд относительно master.

```
git checkout -b feature
git commit -am "feature change"
git log --oneline --graph --all
```

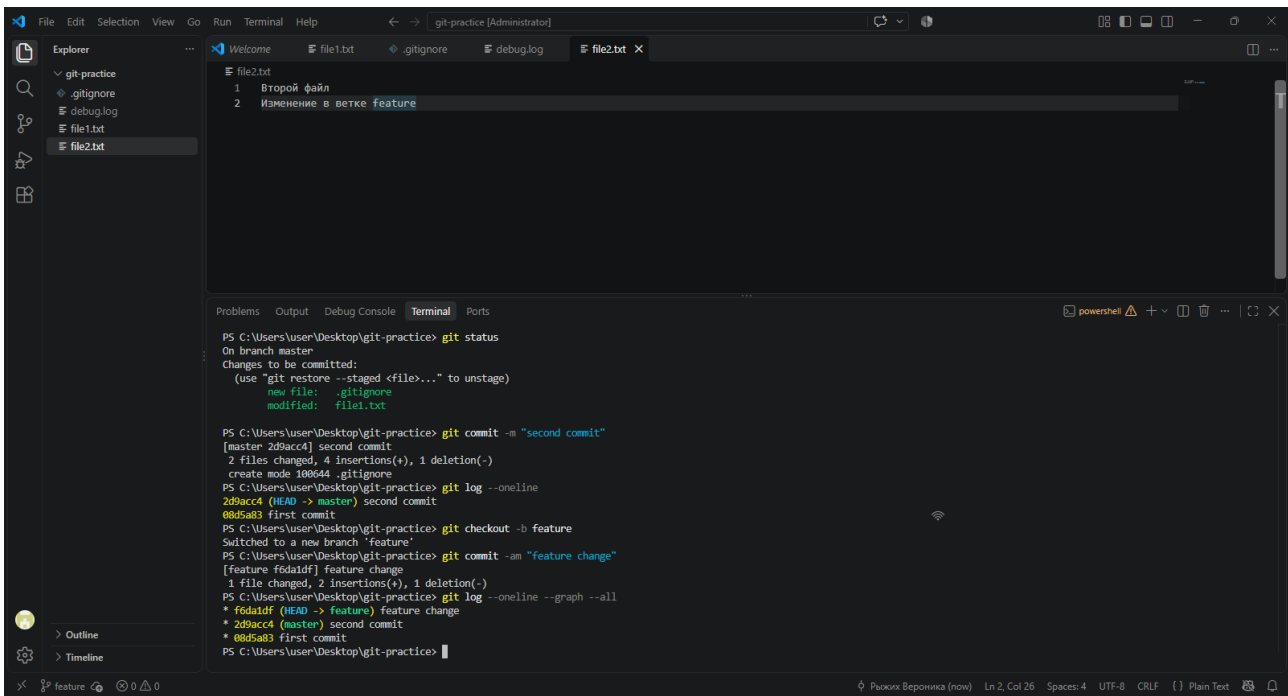
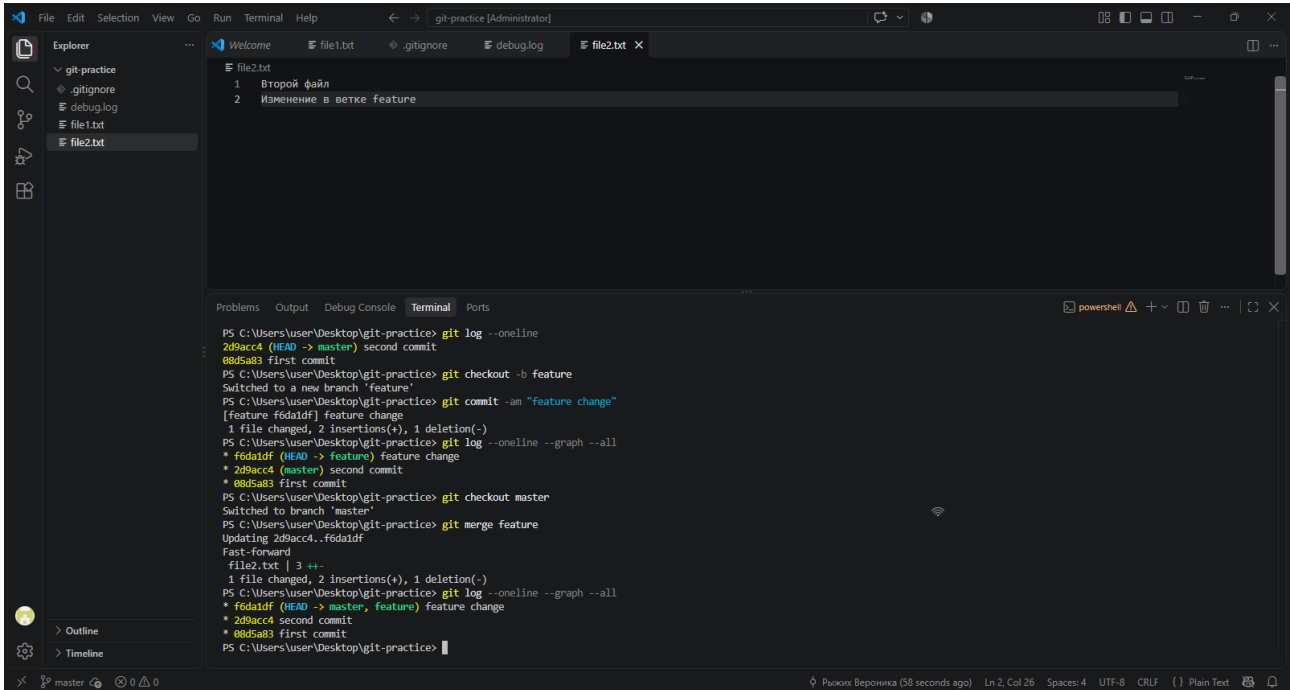


Рис. 10. Создание ветки feature, коммит и граф истории

Шаг 6. Слияние fast-forward

Выполнен возврат в основную ветку master и слияние с feature. Так как master не содержит новых коммитов, которых нет в feature, ветки лежат на одной линии, и Git просто переносит указатель master вперёд. Это слияние fast-forward без коммита слияния и без конфликтов. После него обе ветки указывают на один и тот же коммит.

```
git checkout master
git merge feature
git log --oneline --graph --all
```



```
PS C:\Users\User\Desktop\git-practice> git log --oneline
2d9acc4 (HEAD -> master) second commit
00d5a83 first commit
PS C:\Users\User\Desktop\git-practice> git checkout -b feature
Switched to a new branch 'feature'
PS C:\Users\User\Desktop\git-practice> git commit -am "Feature change"
[feature fd0a1df] feature change
1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\User\Desktop\git-practice> git log --oneline --graph --all
* fd0a1df (HEAD -> feature) feature change
* 2d9acc4 (master) second commit
* 00d5a83 first commit
PS C:\Users\User\Desktop\git-practice> git checkout master
Switched to branch 'master'
PS C:\Users\User\Desktop\git-practice> git merge feature
Updating 2d9acc4..fd0a1df
Fast-forward
 file2.txt | 3 ++-
1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\User\Desktop\git-practice> git log --oneline --graph --all
* fd0a1df (HEAD -> master, feature) feature change
* 2d9acc4 second commit
* 00d5a83 first commit
PS C:\Users\User\Desktop\git-practice>
```

Рис. 11. Слияние fast-forward: master и feature на одном коммите

Шаг 7. Изменение в основной ветке

Находясь в ветке master, в конец file1.txt добавлена строка «Изменение в ветке master», изменение закоммичено. С этого момента истории master и feature расходятся: в каждой будут собственные изменения одного и того же файла.

```
git commit -am "master change"
```

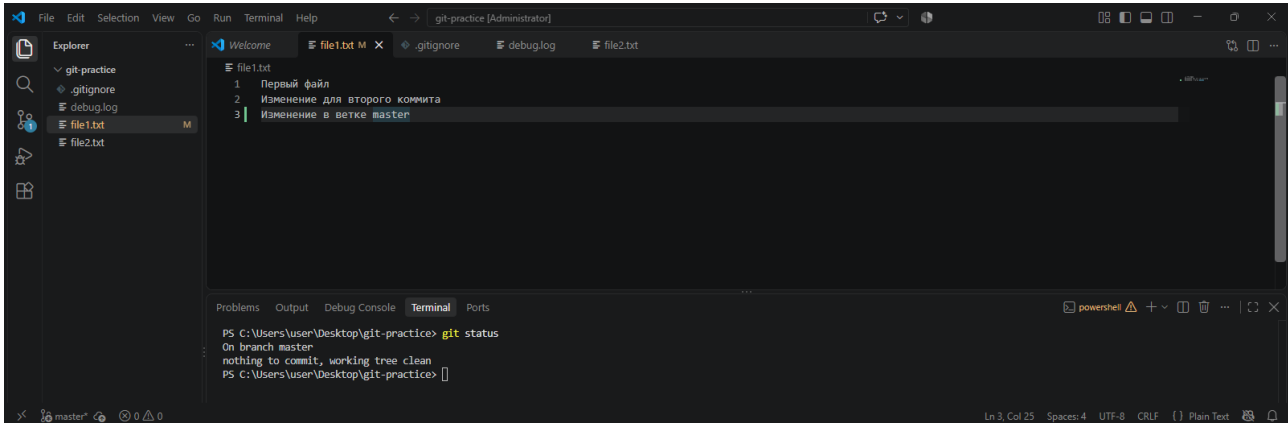


Рис. 12. Изменённый file1.txt в ветке master

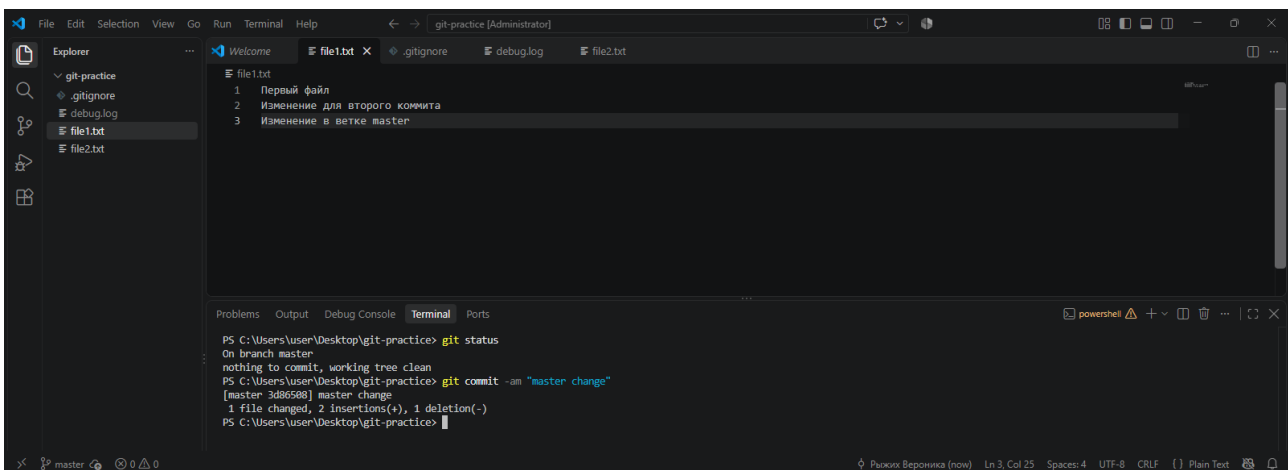


Рис. 13. Коммит «master change»

Шаг 8. Конфликт слияния и его разрешение

В ветке `feature` в тот же файл `file1.txt` дописана другая строка («Изменение в ветке `feature`») и сделан коммит. При слиянии **git merge feature** из `master` Git не смог объединить изменения автоматически: оба коммита правили одно и то же место файла, поэтому возник конфликт (CONFLICT (content)). Git пометил конфликтный участок маркерами `<<<<<<<`, `=====` и `>>>>>>>`. Конфликт разрешён вручную: оставлены обе строки, маркеры удалены. Затем файл добавлен в индекс и создан коммит слияния «`resolve conflict`».

```
git checkout feature
git commit -am "feature change 2"
git checkout master
git merge feature      # CONFLICT
# правка file1.txt вручную
git add file1.txt
git commit -m "resolve conflict"
```

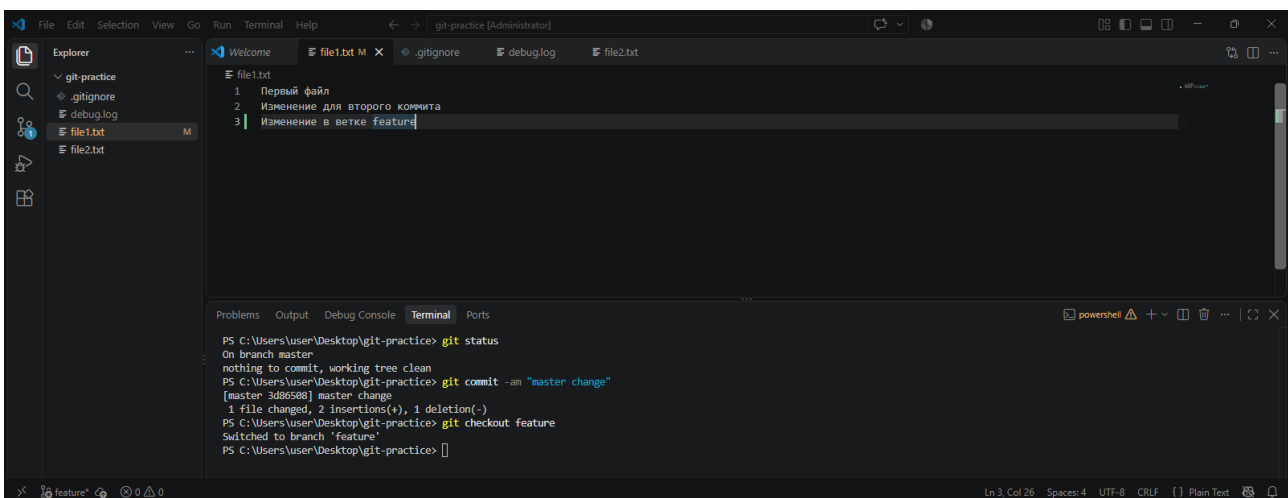


Рис. 14. Ветка `feature`: в `file1.txt` добавлена своя строка

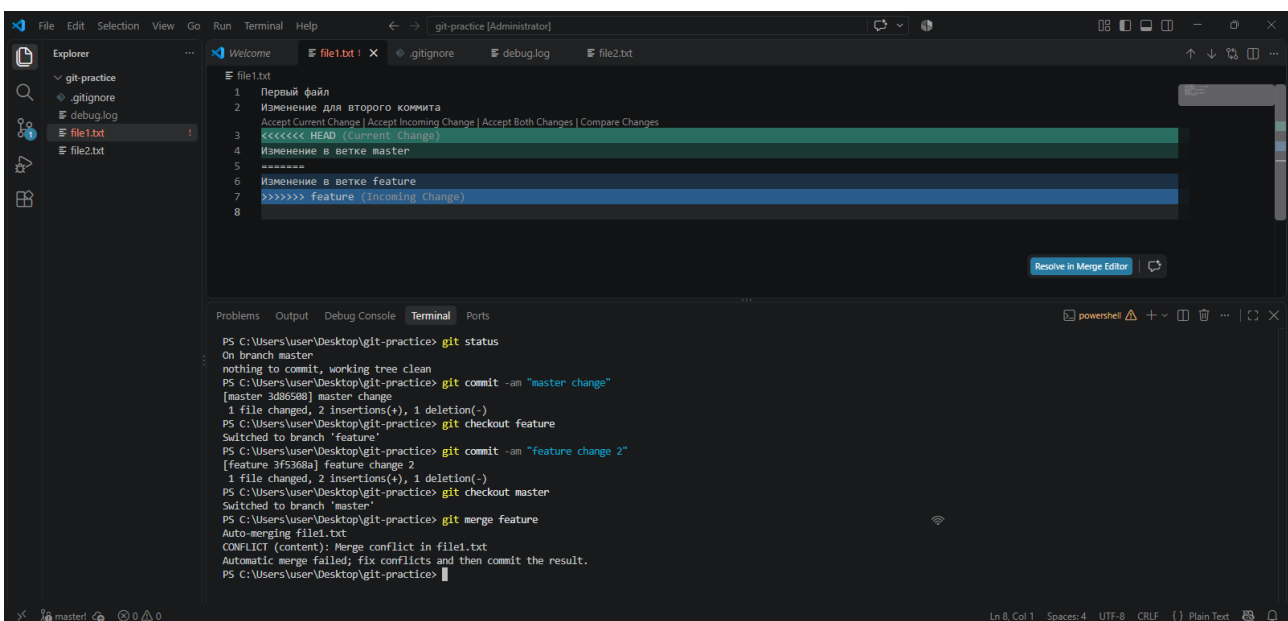


Рис. 15. Конфликт при слиянии: файл с маркерами конфликта

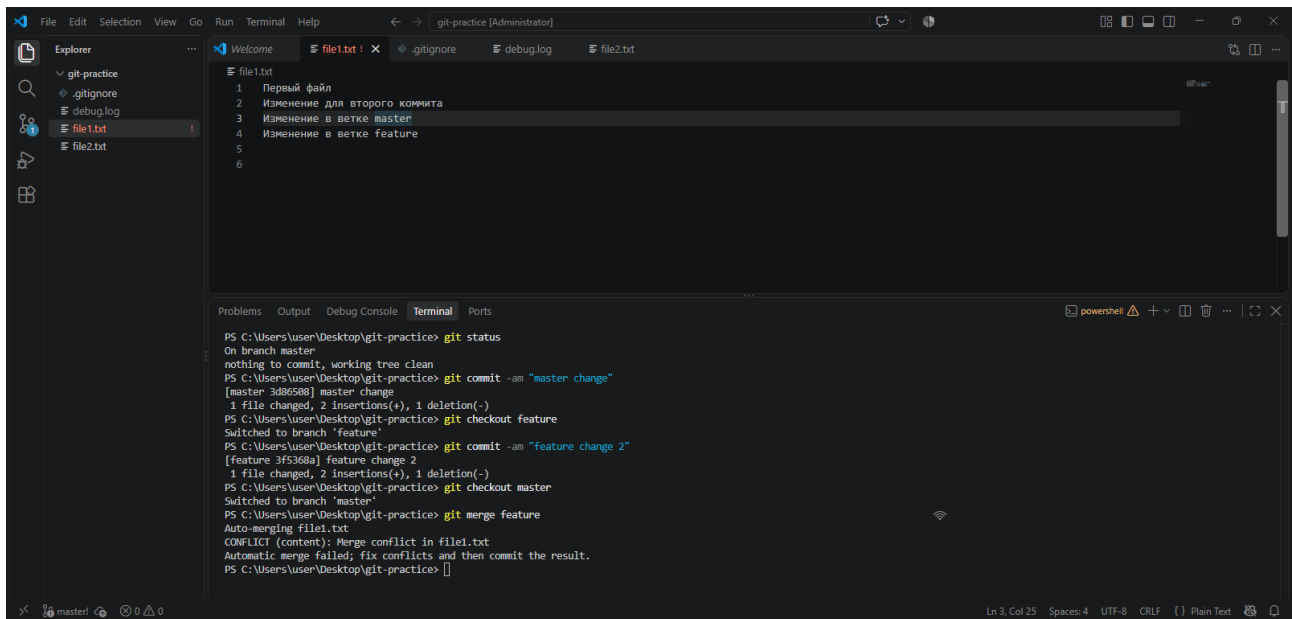


Рис. 16. Файл после ручного разрешения: обе строки, маркеров нет

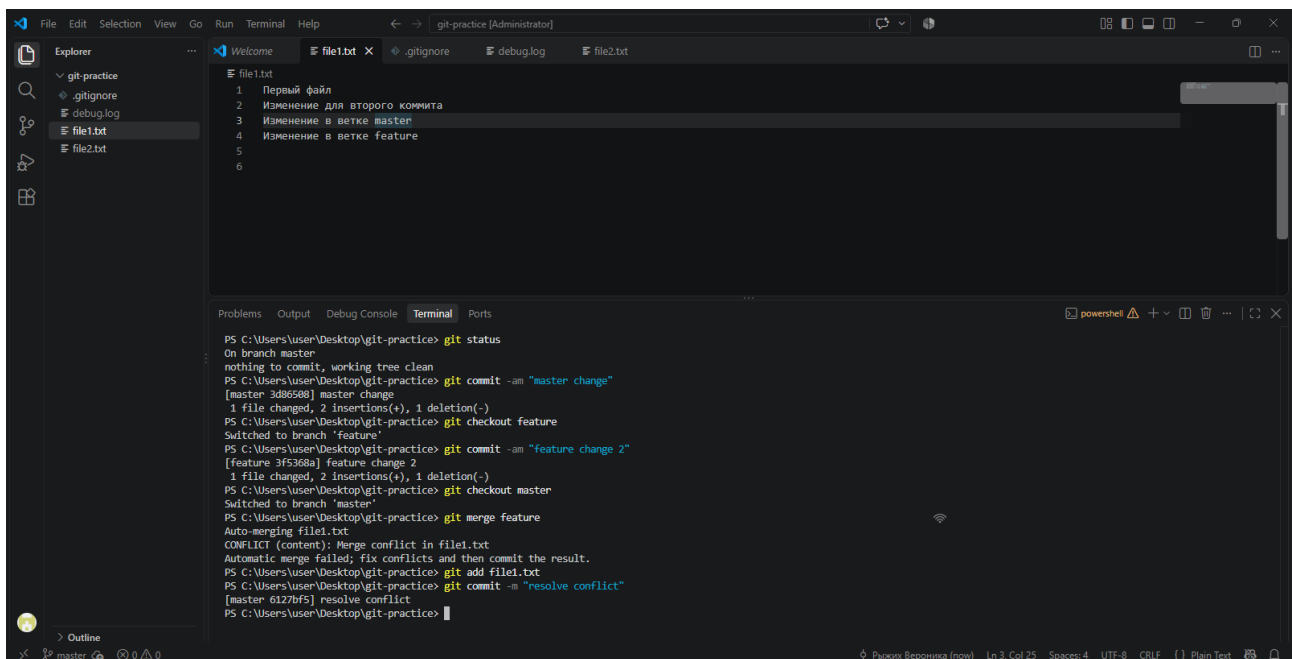


Рис. 17. Файл добавлен в индекс, коммит слияния «resolve conflict»

Шаг 9. История коммитов с ветвлениями

Команда **git log --oneline --graph --all** выводит историю в виде графа. На нём видно, что после общего коммита ветки разошлись («master change» и «feature change 2»), а затем сошлись в коммите слияния «resolve conflict».

```
git log --oneline --graph --all
```

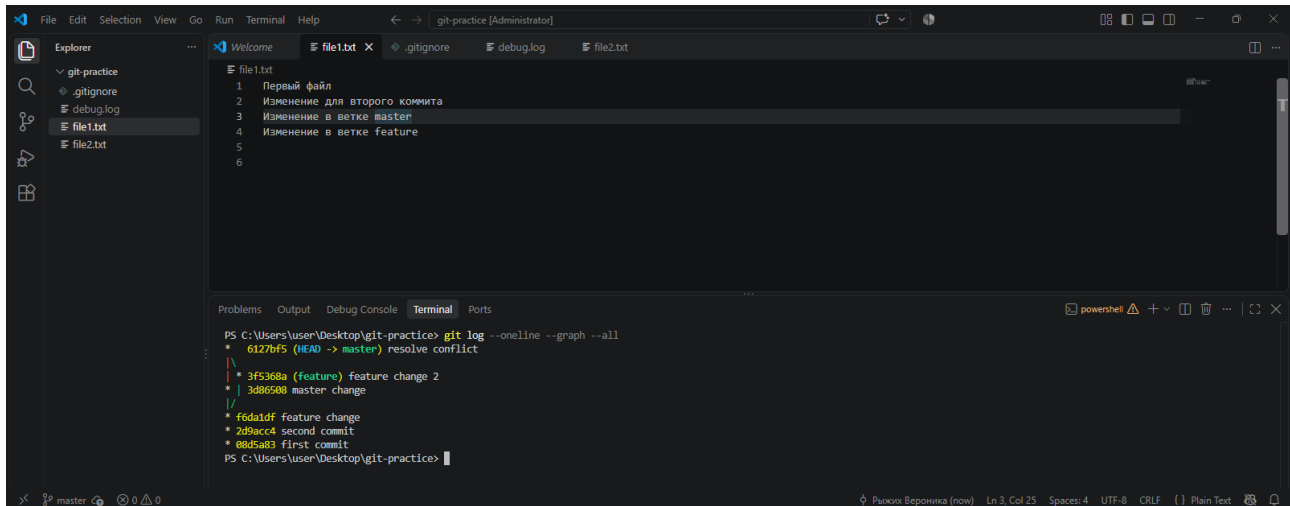


Рис. 18. Граф истории: расхождение и слияние веток

Шаг 10. Удалённый репозиторий на GitHub

На GitHub создан пустой публичный репозиторий `git-practice` (без README и `.gitignore`). Локальный репозиторий связан с ним командой **`git remote add origin`**; **`git remote -v`** подтверждает адреса `fetch` и `push`. Командой **`git push -u origin master`** все коммиты отправлены на сервер, а ветка `master` настроена на отслеживание `origin/master`. На скриншоте `push` стоит уже повторный запуск команды, поэтому в конце «Everything up-to-date»: все коммиты уже были на сервере. На странице репозитория видны файлы проекта, файла `debug.log` нет (он игнорируется). Интерфейс GitHub на скриншотах автоматически переведён браузером.

```
git remote add origin https://github.com/00Veronika00/git-practice.git
git remote -v
git push -u origin master
```

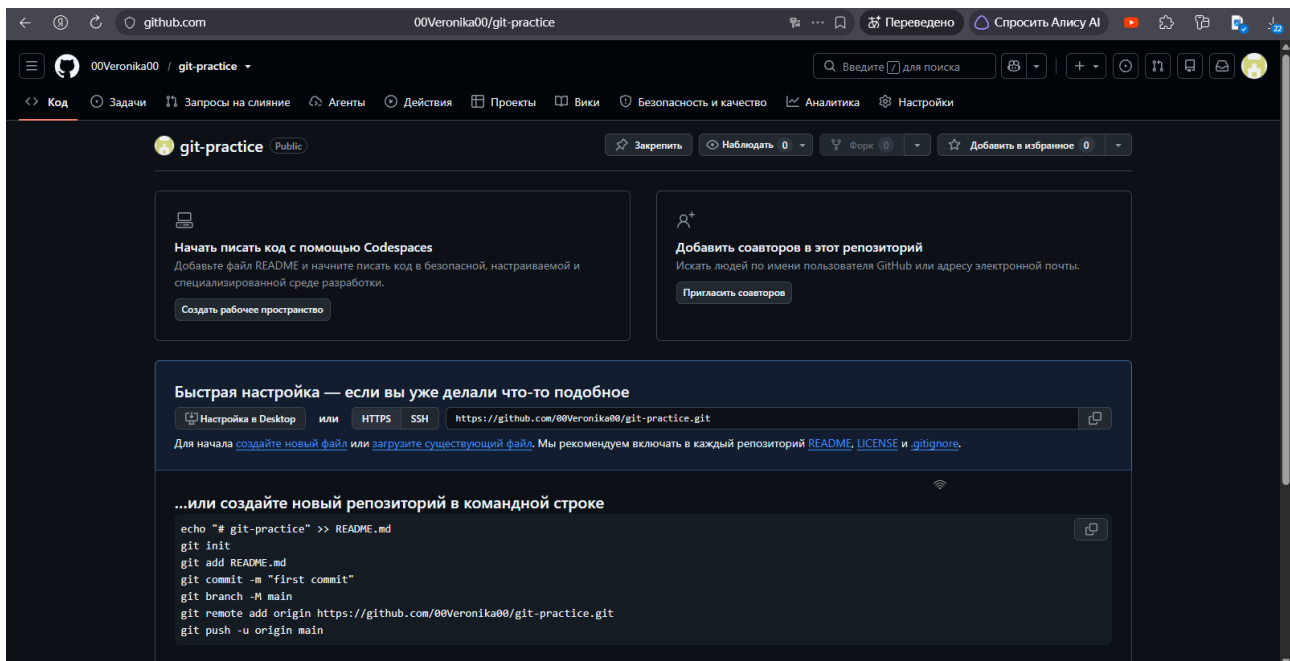


Рис. 19. Создан пустой репозиторий `git-practice` на GitHub

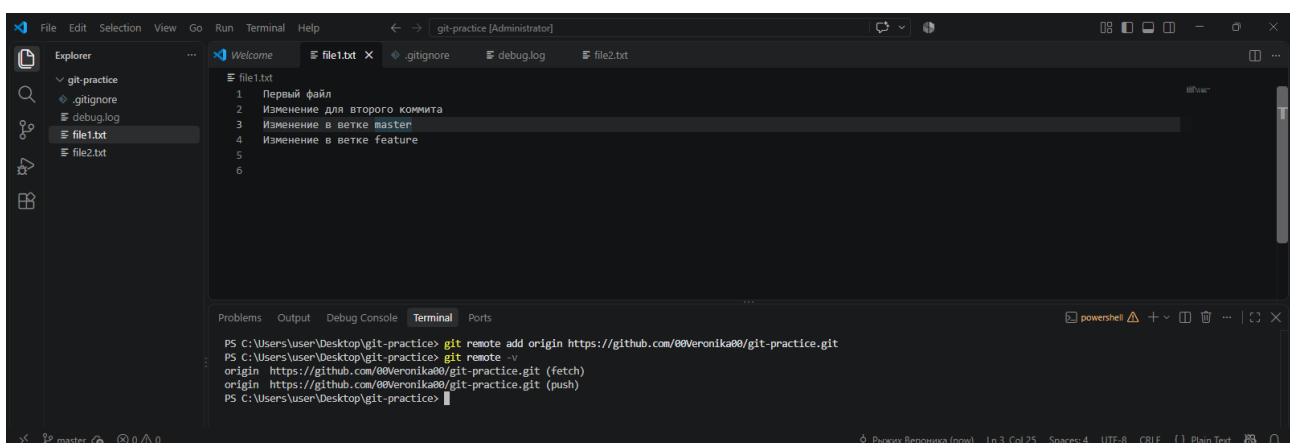


Рис. 20. Репозиторий связан с локальным: `git remote add` и `git remote -v`

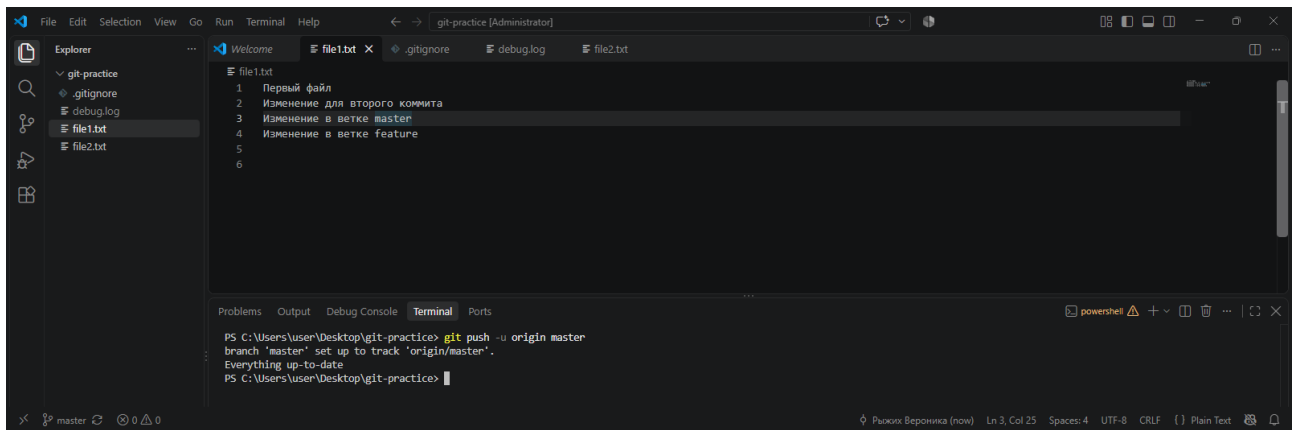


Рис. 21. git push -u origin master: ветка master отслеживает origin/master

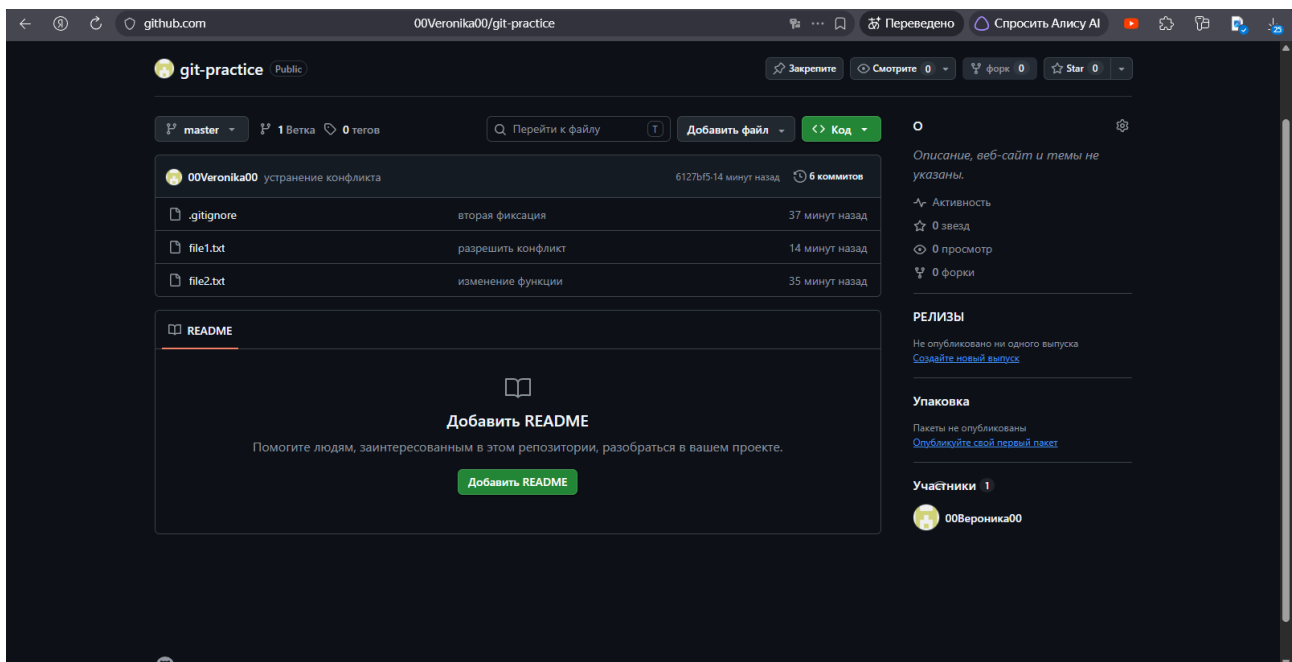


Рис. 22. Репозиторий на GitHub после отправки: файлы и 6 коммитов

Шаг 11. Файл README.md на GitHub

Через веб-интерфейс GitHub (кнопка «Add a README») создан файл README.md с кратким описанием проекта. Он автоматически отображается на главной странице репозитория. Теперь на GitHub 7 коммитов (появился «Create README.md»), а на локальном компьютере этого файла ещё нет.

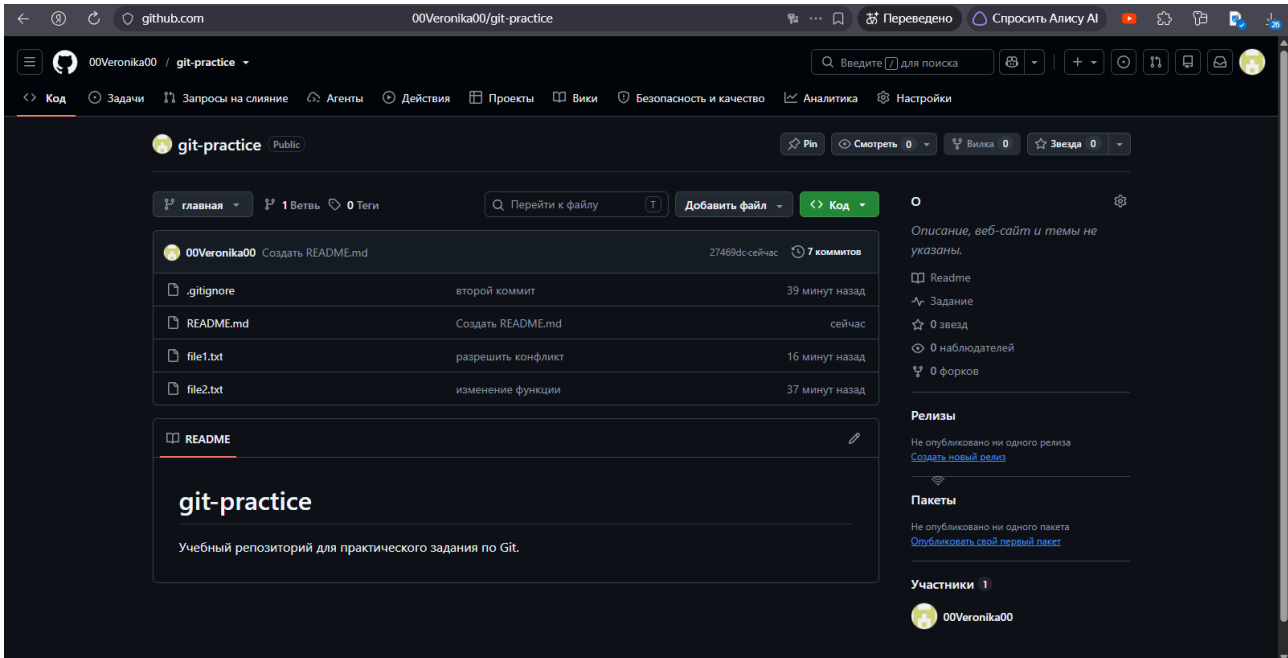
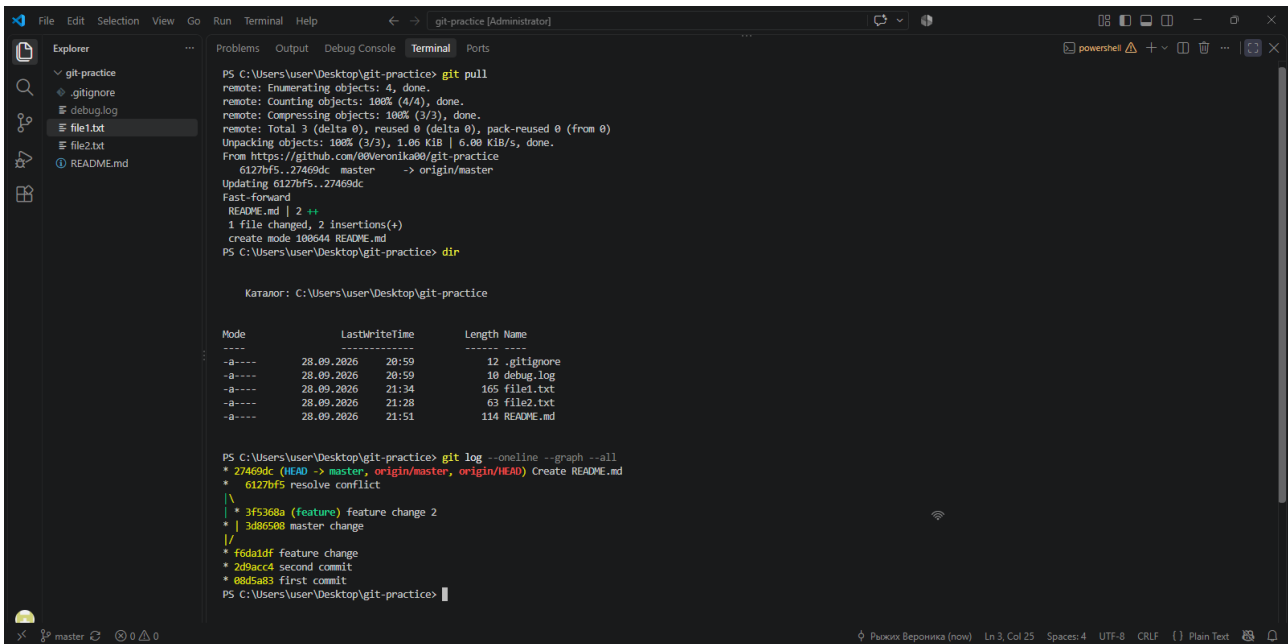


Рис. 23. README.md на главной странице репозитория (7 коммитов)

Шаг 12. Получение изменений с GitHub (git pull)

Командой **git pull** локальный репозиторий получил новый коммит с GitHub. Git сообщил о слиянии fast-forward и создании README.md. Команда **dir** подтверждает, что файл появился в папке, а в логе рядом с master видны указатели origin/master и origin/HEAD: локальная и удалённая ветки синхронизированы.

```
git pull
dir
git log --oneline --graph --all
```



The screenshot shows a Windows terminal window with the following output:

```
PS C:\Users\User\Desktop\git-practice> git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1.00 KiB | 6.00 KiB/s, done.
From https://github.com/00veronika00/git-practice
6127bf5..27469dc master -> origin/master
Updating 6127bf5..27469dc
Fast-forward
 README.md | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 README.md
PS C:\Users\User\Desktop\git-practice> dir

Kataranor: C:\Users\User\Desktop\git-practice

Mode                LastWriteTime         Length Name
-----
-a----          28.09.2026   20:59             12 .gitignore
-a----          28.09.2026   20:59             10 debug.log
-a----          28.09.2026   21:34            165 file1.txt
-a----          28.09.2026   21:28             63 file2.txt
-a----          28.09.2026   21:51            114 README.md

PS C:\Users\User\Desktop\git-practice> git log --oneline --graph --all
* 27469dc (HEAD -> master, origin/master, origin/HEAD) Create README.md
* 6127bf5 resolve conflict
|\
| * 3f5368a (feature) feature change 2
| * | 3d86508 master change
|/
* f6da1df feature change
* 2d9acc4 second commit
* 08d5a83 first commit
PS C:\Users\User\Desktop\git-practice>
```

Рис. 24. git pull, список файлов с README.md и обновлённый лог

Итог

В ходе работы создан локальный репозиторий, выполнены коммиты, настроен .gitignore, освоены ветвление, слияние fast-forward и разрешение конфликта слияния. Репозиторий опубликован на GitHub, создан README.md и получен на компьютер командой git pull. Этот отчёт добавляется в корень репозитория в виде файла report.pdf.